GESTIÓN SCOUT / 01

Un grupo organizado, más tiempo para explorar

Gestión Scout

Informe técnico · Tecnologías Web

Miguel Alejandro Sánchez Pérez

Tecnología asignada: FastAPI (Python)

20 de octubre de 2026 · America/La_Paz

BORRADOR DE ENTREGA · VERSIÓN LOCAL VERIFICADA

Pendiente: confirmar con el docente si acepta ejecución local y realizar el envío en Moodle. No se declara una aplicación publicada en Internet.

Problema y propósito

La información de ramas, integrantes y actividades dispersa en hojas separadas dificulta controlar quién participa y cuántos cupos quedan. El sistema permite a los dirigentes gestionar estos datos con una única interfaz y una base persistente.

Alcance deliberado

Se administra un solo grupo scout, con nombre y ramas configurables. Solo los dirigentes tienen cuenta. Scouts y familiares no inician sesión. Los registros de demostración son ficticios. No se incorporan pagos, mensajería, certificados, adjuntos ni multigrupo.

Escenario de uso

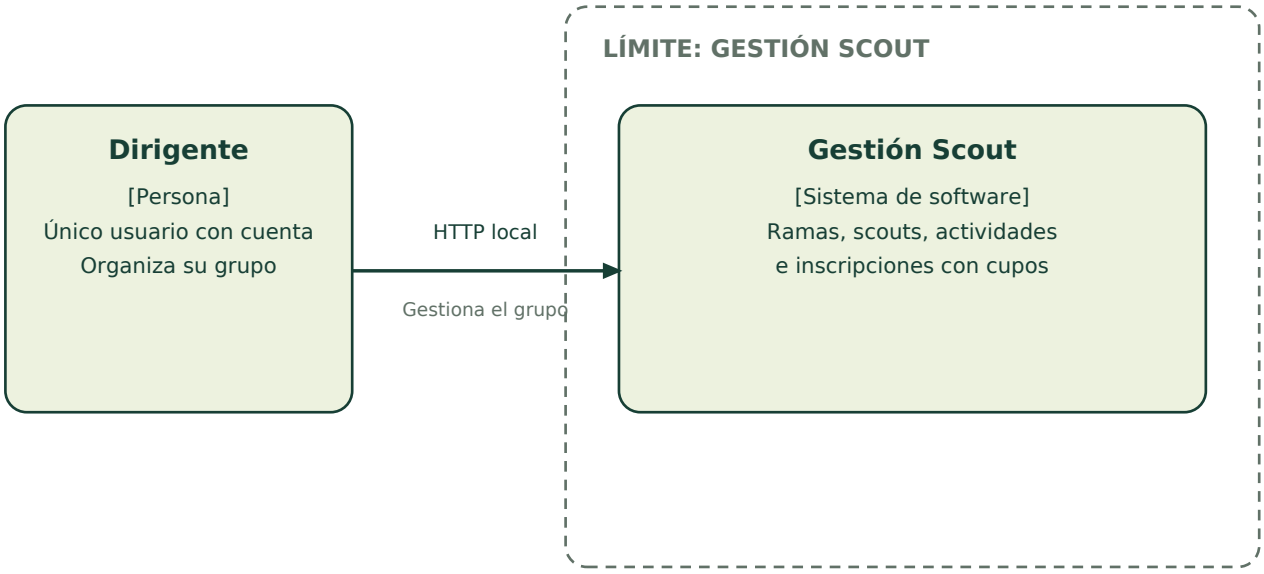
Un dirigente crea una actividad futura abierta, selecciona su rama destinataria o todas las ramas y define cupos. Inscribe scouts activos. Cancelar una inscripción libera un cupo; reactivarla exige cumplir nuevamente las reglas. El resumen muestra la información confirmada por la API.

Componente	Tecnología y responsabilidad
Entrada HTTP	Nginx: frontend, proxy inverso, trazas y límites
Negocio	FastAPI + Pydantic + SQLAlchemy + Alembic
Persistencia	PostgreSQL, claves foráneas y volumen nombrado
Interfaz	HTML, CSS y JavaScript sin compilación

GESTIÓN SCOUT / 02

C4 · Nivel 1: contexto

El contexto presenta la persona que utiliza el sistema y su objetivo. La frontera encapsula la solución completa: los detalles de Nginx, FastAPI y PostgreSQL se desarrollan en el siguiente nivel.



Persona y límite de responsabilidad

Dirigente: único actor con cuenta; administra ramas, scouts, actividades e inscripciones. Su contraseña se verifica en el backend y su acceso requiere un JWT vigente. No existe registro público.

Gestión Scout: sistema de software responsable de mantener datos consistentes y controlar cupos. Los scouts son sujetos del dominio cuyos datos administra el dirigente; no se representan como clientes autenticados ni servicios externos.

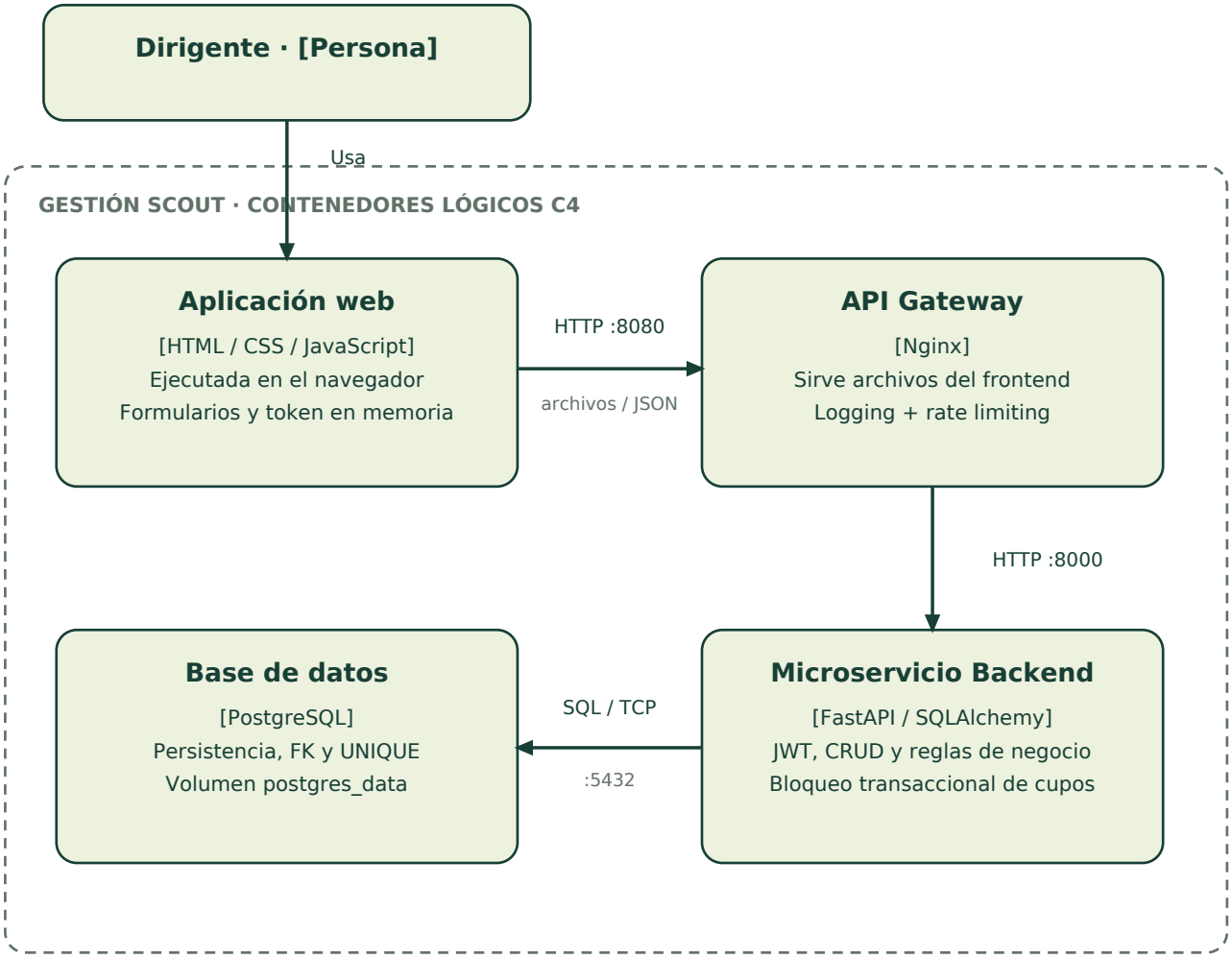
Relación y entorno

El dirigente usa un navegador que accede por HTTP local al gateway. El host escucha en 127.0.0.1:8080. La ejecución en este equipo corresponde al alcance acordado por el autor; no hay dependencia de cuentas de nube ni exposición pública de PostgreSQL.

Fuentes editables: docs/diagramas/contexto.mmd y la función context() de scripts/generar_informe.py. Versiones renderizadas: contexto.svg y contexto.pdf.

GESTIÓN SCOUT / 03

C4 · Nivel 2: contenedores



C4 describe cuatro contenedores lógicos: interfaz, gateway, backend y base de datos. Compose ejecuta tres contenedores Docker: gateway, backend y db. El frontend corre en el navegador; sus archivos están incluidos en la imagen de Nginx. No necesita un cuarto servicio Docker.

Comunicación y orquestación

Solo Nginx publica un puerto del host. La API y Swagger se reenvían sin quitar prefijos: /api/v1, /docs y /openapi.json. Backend usa db:5432 dentro de la red bridge. PostgreSQL dispone de volumen nombrado para conservar información al recrear contenedores.

depends_on espera la salud de la base antes del backend y del backend antes del gateway. La disponibilidad de FastAPI ejecuta SELECT 1. El healthcheck del gateway consulta ese endpoint mediante el proxy; no se limita a comprobar un proceso.

Fuentes: docker-compose.yml, Dockerfiles, gateway/default.conf.template y docs/diagramas/contenedores.mmd. SVG y PDF vectoriales incluidos.

GESTIÓN SCOUT / 04

Modelo, seguridad y transacciones

Entidad	Campos esenciales y relaciones
Rama	id, nombre único, descripción
Scout	id, nombres, apellidos, rama_id, activo
Actividad	id, título, descripción, lugar, inicio/fin con zona, cupo, rama opcional, estado
Inscripción	id, scout_id, actividad_id, fecha_registro, estado; UNIQUE(scout, actividad)
Dirigente	id, nombre, usuario único, hash de contraseña, activo

DTO, Repository e inyección de dependencias

Pydantic valida DTO de entrada y limita las salidas: nunca expone password_hash. Los routers traducen HTTP; los repositorios concentran consultas; GestionService aplica reglas. Depends(get_db) entrega una sesión por solicitud y Depends(current_user) exige una cuenta activa. Alembic incorpora la revisión 001 antes de iniciar Uvicorn.

Último cupo: validación transaccional

Crear o modificar inscripciones bloquea primero la fila de la actividad con SELECT FOR UPDATE. Dentro de la misma transacción se valida scout activo, rama, estado abierto, inicio futuro y cantidad inscrita. Con READ COMMITTED, la solicitud que esperó cuenta los cambios confirmados: no alcanza con contar fuera del bloqueo. Actualizar cupos usa el mismo bloqueo; cambiar destinatarios también bloquea scouts inscritos.

Protección y fallos comprensibles

Contraseñas Argon2 y JWT HS256 con sub, iat y exp. El token dura 30 minutos y queda solo en memoria de la pestaña. Nginx genera X-Request-ID y limita la API a 20r/s y login a 5r/m. No registra cuerpos ni Authorization. Las claves foráneas RESTRICT conservan referencias; conflictos retornan 409. No hay cascadas inesperadas ni registro público.

Cancelar o eliminar libera cupo. Reactivar exige las mismas reglas. Reducir cupo por debajo de los inscritos o cambiar a una rama incompatible devuelve 409. Fechas sin zona, cupos no positivos y campos vacíos generan 422.

GESTIÓN SCOUT / 05

Contrato REST y ejemplos JSON

Base local: http://localhost:8080/api/v1. Todos los recursos de negocio requieren Authorization: Bearer. Los GET de colección aceptan los filtros indicados. PUT reemplaza el DTO del recurso; en inscripciones solo cambia estado.

Métodos	Ruta relativa a /api/v1	Cuerpo / filtro
POST	/auth/login	Formulario username, password
GET	/auth/me · /resumen	Cuenta sin hash / cantidades
GET · POST	/ramas	RamaIn en POST
GET · PUT · DELETE	/ramas/{id}	RamaIn en PUT
GET · POST	/scouts	ScoutIn; filtro rama_id
GET · PUT · DELETE	/scouts/{id}	ScoutIn en PUT
GET · POST	/actividades	ActividadIn; rama_id incluye generales
GET · PUT · DELETE	/actividades/{id}	ActividadIn en PUT
GET · POST	/inscripciones	scout_id, actividad_id, estado; filtros por ambos IDs
GET · PUT · DELETE	/inscripciones/{id}	En PUT: estado inscrito / cancelado

```
RamaIn: {"nombre":"Exploradores","descripcion":"Ejemplo"}
ScoutIn: {"nombres":"Scout","apellidos":"Ficticio",
        "rama_id":1,"activo":true}
ActividadIn: {"titulo":"Salida","descripcion":"Ejemplo",
  "lugar":"Parque","inicio":"2030-01-20T09:00:00-04:00",
  "fin":"2030-01-20T12:00:00-04:00","cupo_maximo":10,
  "rama_id":1,"estado":"abierta"}
InscripcionIn: {"scout_id":1,"actividad_id":1,"estado":"inscrito"}
PUT inscripcion: {"estado":"cancelado"}
```

Respuestas: GET/PUT 200; POST de recurso 201 con id; DELETE 204 sin cuerpo. ActividadOut agrega inscritos y cupos_disponibles; InscripcionOut agrega id y fecha_registro. Login devuelve access_token, token_type y expires_in.

Errores: 401 acceso, 404 referencia, 409 conflicto de regla/BD, 422 validación y 429 Nginx. El esquema detallado de todos los DTO está exportado en docs/evidencias/openapi.json y disponible en /docs. Las fechas/IDs de estos JSON son ejemplos; seed calcula fechas futuras al ejecutarse.

GESTIÓN SCOUT / 06

Pruebas reales y estado de Docker

Resultado registrado: 17 passed in 3.59s. PostgreSQL real, backend en Docker y solicitudes al gateway del proyecto aislado scout-tests. No se utilizó SQLite ni se simuló el motor.

Grupo de pruebas	Comprobación
Acceso y contrato	Login válido/incorrecto; token ausente, inválido y expirado; OpenAPI y /health
CRUD y datos	Crear, leer, actualizar y eliminar las cuatro entidades; filtros y validación 422
Reglas de inscripción	Duplicado, cupo lleno, cancelar/reactivar, rama, scout inactivo, fecha y estado
Concurrencia	Dos solicitudes al último cupo: exactamente un HTTP 201 y un HTTP 409
Integridad y gateway	FK al borrar; reducción de cupo; cambios de rama incompatibles; rate limit 429

Extracto del estado real de contenedores

```
scout-tests-backend-1  healthy
scout-tests-db-1  healthy
scout-tests-gateway-1  healthy
Puertos publicados de prueba: 127.0.0.1:18080 -> gateway:80
```

La primera construcción del proyecto de pruebas aplicó Alembic 001 sobre su volumen vacío. Tras down y up --wait, se verificó login HTTP 200 con la cuenta creada antes del reinicio: el volumen preservó el registro. Los tres servicios recuperaron estado healthy.

Reproducir y auditar

```
python3 -m venv .venv
.venv/bin/pip install -r tests/requirements.txt
.venv/bin/python scripts/probar.py
```

Evidencia completa: resultados.txt, estado-docker.txt y docker-verificacion.txt. Las fixtures eliminan sus propios registros; el proyecto de demostración gestion-scout no se usa para estas pruebas. No se ejecutó Postman.

GESTIÓN SCOUT / 07

Evidencia de interfaz, Swagger y trazas

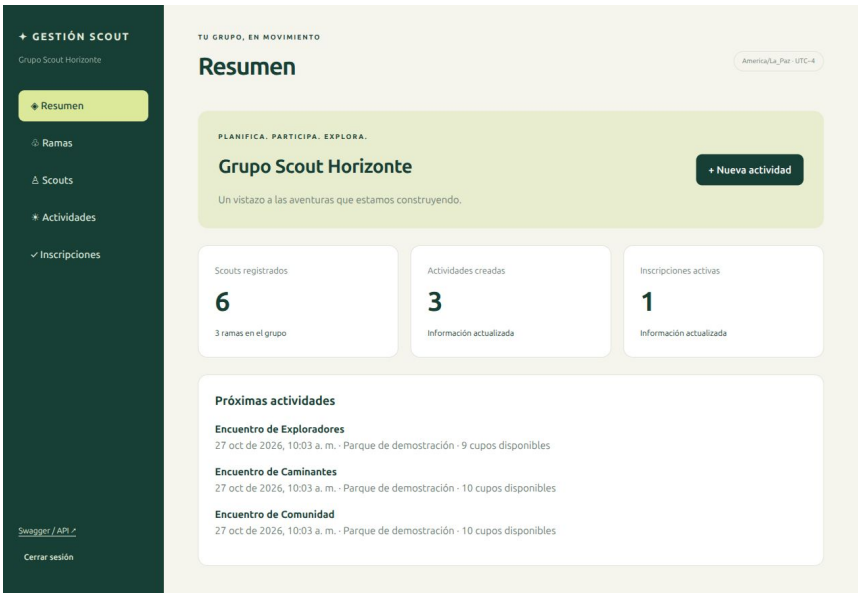


Figura 1. Captura real del resumen local después de crear, cancelar y reactivar una inscripción desde la página. Se muestran seis scouts ficticios, tres actividades y una inscripción activa.

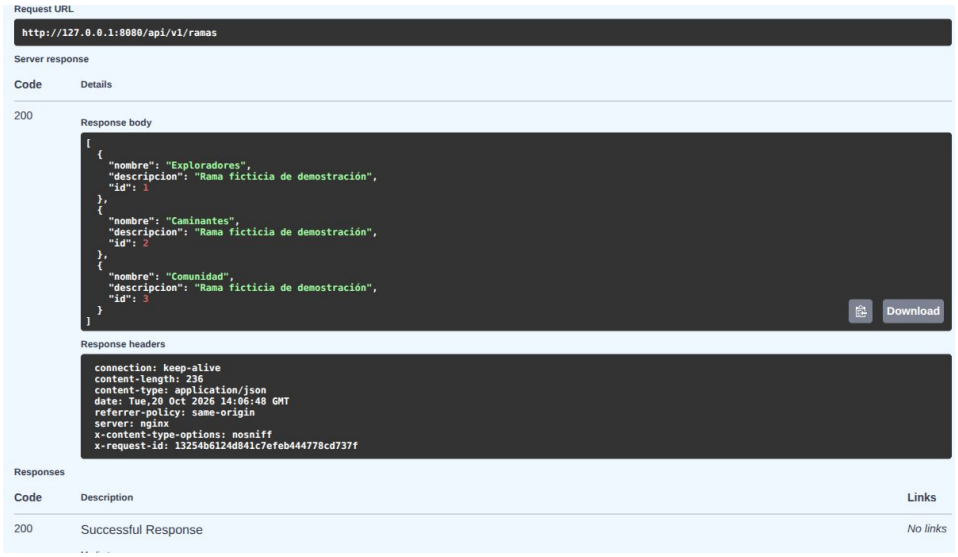


Figura 2. Swagger detrás de Nginx: GET /api/v1/ramas autenticado, HTTP 200, JSON real y X-Request-ID. El recorte excluye el bloque cURL que contiene el token.

Logs correlacionados y límite

POST /api/v1/ramas -> 201; 0.017 s
request_id=aaf4e433efcb53e89029716a4b52d26e (Nginx / FastAPI)
POST /api/v1/auth/login -> 429

Los logs reales en gateway-extracto.log contienen method, path, status y duration. El mismo request ID llega a FastAPI. Las solicitudes repetidas de login produjeron HTTP 429 en Nginx. Los logs completos se conservan localmente; no se publican contraseñas, cuerpos ni tokens.

GESTIÓN SCOUT / 08

Entrega local, rúbrica y pendientes

Para una clonación limpia: generar .env con scripts/configurar.py; ejecutar docker compose up --build; crear dirigente con app.cli create-dirigente; generar ejemplos con app.cli seed; abrir localhost:8080. Solo gateway publica un puerto y está ligado a loopback. El repositorio público distribuye código: no ejecuta los contenedores.

Criterio	Archivos y evidencia	Estado
Negocio + C4 (20)	README, diagramas, informe pp. 1-3	Documentado / renderizado
Gateway (25)	gateway/, pruebas 02/99, logs	Logging + 429 comprobados
Backend + BD (30)	app/, Alembic, pruebas CRUD / cupos	17 pruebas aprobadas
Docker (15)	Compose, Dockerfiles, evidencia de reinicio	Bridge, salud y persistencia
Git / README / Moodle (10)	feature/*, release/*, README, v1.0.0	Código y guía; Moodle pendiente

Repositorio y flujo de trabajo

Repositorio: github.com/migusaper1530/gestion-scout-tecnoweb. Se inició main, se desarrolló en ramas feature/* sobre develop, y la entrega se estabiliza en release/1.0.0 antes de integrarse a main y etiquetarse. El historial contiene trabajo real por etapas, sin commits vacíos para simular Git Flow.

Pendientes y limitaciones verificables

El autor solicitó ejecución local y no alojamiento web. No hay URL pública de aplicación ni pruebas HTTPS externas. Si el docente exige un servidor independiente, ese punto necesita su validación. El alumno debe revisar el PDF y subir el enlace y el informe a Moodle; el asistente no realizó ese envío. Por estos pasos externos se mantiene la marca de borrador de entrega.

Swagger utiliza los assets CDN predeterminados de FastAPI; requiere conexión para cargarlos. La interfaz propia y la API se sirven localmente. Las evidencias de Docker del informe son extractos textuales de ejecuciones auténticas; una captura de terminal adicional puede tomarse con los comandos de docs/evidencias/README.md.

Conclusión

La arquitectura local satisface la separación gateway / microservicio / base de datos, protege el acceso y conserva integridad bajo concurrencia. La demostración puede ejecutarse sin una cuenta de nube. El estudiante dispone de fuente legible, pruebas reproducibles, diagramas y guía de defensa para explicar cada decisión.

Referencias oficiales: fastapi.tiangolo.com/tutorial/security/oauth2-jwt/ · nginx.org/en/docs/http/ngx_http_limit_req_module.html · docs.sqlalchemy.org/en/20/ · docs.docker.com/compose/ · c4model.com/diagrams